

NiceGUI ui.dark_mode 完全指南（基于官方文档深度解析）

`ui.dark_mode` 是 NiceGUI 提供的原生暗黑模式管理工具，支持全局暗黑模式的启用 / 禁用、动态切换、响应式适配系统主题，以及与自定义样式的深度集成。其核心价值在于**简化暗黑模式开发流程**，无需手动编写大量明暗样式对比代码，即可实现全局 UI 主题的无缝切换，尤其适合图书管理系统等需要兼顾不同使用场景（日间 / 夜间办公）的应用。

一、核心定义与设计理念

1. 本质与定位

- 类型：**`ui.dark_mode` 是一个「响应式布尔变量」（继承自 `nicegui.reactive.Value[bool]`），同时提供配套的控制方法。
- 核心作用：**
 - 全局控制 NiceGUI/Quasar 组件的暗黑模式样式（如按钮、卡片、输入框等自动适配颜色）；
 - 响应系统主题（自动检测用户设备的暗黑模式偏好）；
 - 支持运行时动态切换，触发 UI 实时更新；
 - 可被组件 / 样式监听，实现自定义暗黑模式适配。
- 底层原理：**
 - 启用时，会为 HTML 根元素添加 `dark` 类，并设置 Quasar 主题变量（如 `--q-dark`）；
 - NiceGUI 内置组件会根据 `dark` 类自动切换预设的明暗样式；
 - 自定义样式可通过 `:root.dark` 选择器适配暗黑模式。

2. 与手动实现暗黑模式的区别

特性	<code>ui.dark_mode</code> 方式	手动实现方式（自定义 CSS）
开发成本	零配置启用，组件自动适配	需手动编写两套样式（明 / 暗），维护成本高
组件兼容性	完美兼容所有 NiceGUI/Quasar 内置组件	需为每个组件 / 自定义样式单独适配
动态切换	一行代码切换，UI 实时响应	需手动切换 CSS 类 / 变量，代码冗余
系统主题适配	原生支持 <code>follow_system</code> ，自动同步设备设置	需通过 JS 检测系统主题，手动实现同步
扩展性	支持监听变量变化，自定义适配逻辑	需手动维护样式优先级和切换逻辑

二、基础用法：快速启用与控制

1. 直接启用 / 禁用（静态配置）

通过赋值 `True / False` 直接控制暗黑模式，通常在应用初始化时设置。

```
from nicegui import ui

# 启用暗黑模式（全局生效）
ui.dark_mode.value = True

# 测试组件（自动适配暗黑模式）
ui.label("图书管理系统").classes("text-2xl font-bold")
ui.card().classes("w-64 p-6").add(ui.text("暗黑模式下的卡片"))
ui.input("搜索图书", placeholder="输入书名")
ui.button("添加图书")

ui.run()
```

2. 动态切换（按钮控制）

利用 `ui.dark_mode` 的响应式特性，通过按钮点击切换状态，UI 会实时更新。

```
from nicegui import ui

# 切换暗黑模式的按钮（直接绑定变量）
ui.switch("暗黑模式", value=ui.dark_mode, on_change=lambda e:
ui.dark_mode.set_value(e.value))

# 或通过按钮点击事件切换
def toggle_dark_mode():
    ui.dark_mode.value = not ui.dark_mode.value

ui.button("切换暗黑模式", on_click=toggle_dark_mode)

# 测试组件
with ui.row().classes("gap-4 p-6"):
    ui.input("用户名")
    ui.button("登录")

ui.run()
```

3. 跟随系统主题（自动适配设备设置）

通过 `ui.dark_mode.follow_system()` 让应用主题自动同步用户设备的暗黑模式偏好（如 Windows、macOS、手机系统的深色模式）。

```

from nicegui import ui

# 自动跟随系统主题（优先使用系统设置，可被手动切换覆盖）
ui.dark_mode.follow_system()

# 可选：添加手动切换开关（覆盖系统设置）
ui.switch("暗黑模式", value=ui.dark_mode, on_change=lambda e:
ui.dark_mode.set_value(e.value))

ui.label("自动适配系统主题的应用")
ui.card().add(ui.text("系统暗黑模式开启时，我会自动变暗"))

ui.run()

```

关键说明：

- `follow_system()` 会在应用启动时检测系统主题，并设置 `ui.dark_mode.value`；
- 手动切换开关后，会暂停跟随系统（直到再次调用 `follow_system()`）；
- 支持大多数现代浏览器和操作系统（Windows 10+、macOS 10.14+、iOS 13+、Android 10+）。

三、核心特性：响应式监听与自定义适配

1. 监听暗黑模式变化（自定义逻辑）

由于 `ui.dark_mode` 是响应式变量，可通过 `@ui.reactive` 装饰器或 `watch` 方法监听其变化，执行自定义逻辑（如修改颜色变量、更新数据展示等）。

示例 1：监听变化并提示

```

from nicegui import ui

@ui.reactive
def on_dark_mode_change():
    """暗黑模式变化时触发的逻辑"""
    mode = "暗黑模式" if ui.dark_mode.value else "日间模式"
    ui.notify(f"已切换至 {mode}", type="info")

# 初始化时触发一次
on_dark_mode_change()

# 切换按钮
ui.switch("暗黑模式", value=ui.dark_mode)

ui.run()

```

示例 2：动态修改全局颜色变量

```
from nicegui import ui

# 监听暗黑模式变化，修改 CSS 变量
def update_theme_variables():
    if ui.dark_mode.value:
        # 暗黑模式：深色背景、浅色文本
        ui.add_css('''
            :root {
                --app-background: #1e293b;
                --app-text: #f8fafc;
            }
        ''')
    else:
        # 日间模式：浅色背景、深色文本
        ui.add_css('''
            :root {
                --app-background: #f8fafc;
                --app-text: #1e293b;
            }
        ''')

# 初始化时设置一次
update_theme_variables()
# 监听变化并更新
ui.dark_mode.watch(update_theme_variables)

# 应用自定义变量的组件
ui.add_css('''
    body {
        background-color: var(--app-background);
        color: var(--app-text);
    }
''')

ui.label("使用自定义主题变量的文本")
ui.switch("暗黑模式", value=ui.dark_mode)

ui.run()
```

2. 自定义样式适配（CSS 层面）

通过 `:root.dark` 选择器为自定义组件 / 样式编写暗黑模式专属规则，优先级高于默认样式。

示例：自定义卡片的暗黑模式适配

```
from nicegui import ui

# 定义支持暗黑模式的自定义样式
ui.add_css('''
    /* 日间模式：白色背景、灰色边框 */
```

```

        .custom-card {
            background-color: #ffffff;
            border: 1px solid #e5e7eb;
            border-radius: 8px;
            padding: 16px;
        }
        /* 暗黑模式: 深色背景、深灰色边框 */
        :root.dark .custom-card {
            background-color: #334155;
            border: 1px solid #475569;
            color: #f8fafc;
        }
        /* 暗黑模式下的按钮样式覆盖 */
        :root.dark .custom-btn {
            background-color: #4f46e5;
            color: white;
        }
    ''')

# 切换按钮
ui.switch("暗黑模式", value=ui.dark_mode)

# 应用自定义样式的组件
with ui.card(classes="custom-card"):
    ui.label("自定义卡片标题").classes("font-bold text-lg")
    ui.text("支持暗黑模式自动切换")
    ui.button("自定义按钮", classes="custom-btn")

ui.run()

```

3. 组件级适配（基于暗黑模式状态）

在组件渲染时，可直接根据 `ui.dark_mode.value` 动态调整组件属性（如颜色、图标、文本）。

```

from nicegui import ui

def get_button_color():
    """根据暗黑模式返回不同按钮颜色"""
    return "indigo-500" if ui.dark_mode.value else "blue-500"

# 响应式更新按钮颜色
@ui.reactive
def update_button():
    btn.classes(remove="bg-blue-500 bg-indigo-500")
    btn.classes(add=f"bg-{get_button_color()} text-white")

# 初始化按钮
btn = ui.button("自适应颜色按钮")
update_button()

# 监听暗黑模式变化并更新
ui.dark_mode.watch(update_button)

```

```
# 切换开关
ui.switch("暗黑模式", value=ui.dark_mode)

ui.run()
```

四、高级场景：集成其他样式特性

1. 与 `ui.colors` 配合（统一主题颜色）

`ui.colors` 配置的全局颜色变量可结合暗黑模式，实现更精细的主题控制。

```
from nicegui import ui

def update_global_colors():
    if ui.dark_mode.value:
        # 暗黑模式：深色主题色
        ui.colors(
            primary="#818cf8",
            background="#1e293b",
            surface="#334155",
            text="#f8fafc",
            text_secondary="#94a3b8"
        )
    else:
        # 日间模式：浅色主题色
        ui.colors(
            primary="#3b82f6",
            background="#f8fafc",
            surface="#ffffff",
            text="#1e293b",
            text_secondary="#64748b"
        )

# 初始化与监听
update_global_colors()
ui.dark_mode.watch(update_global_colors)

# 测试组件
ui.label("图书管理系统").classes("text-2xl font-bold")
with ui.row().classes("gap-4 p-6"):
    ui.button("添加图书")
    ui.button("删除图书", color="negative")
    ui.button("修改信息", secondary=True)

ui.switch("暗黑模式", value=ui.dark_mode)

ui.run()
```

2. 与 CSS Layers 配合（样式优先级控制）

当自定义暗黑模式样式需要覆盖 Quasar 内置样式时，可结合 CSS Layers 提升优先级。

```
from nicegui import ui

# 高优先级覆盖 Quasar 按钮样式（暗黑模式）
ui.add_css('''
    @layer utilities {
        /* 日间模式按钮 */
        .custom-btn {
            background-color: #3b82f6 !important;
            color: white !important;
        }
        /* 暗黑模式按钮（高优先级） */
        :root.dark .custom-btn {
            background-color: #818cf8 !important;
            box-shadow: 0 0 0 2px rgba(129, 140, 248, 0.5) !important;
        }
    }
''')

ui.button("高优先级自定义按钮", classes="custom-btn")
ui.switch("暗黑模式", value=ui.dark_mode)

ui.run()
```

3. 持久化暗黑模式设置（本地存储）

通过 `localStorage` 保存用户的暗黑模式偏好，下次启动应用时自动恢复（结合 NiceGUI 的 `ui.run_javascript` 实现）。

```
from nicegui import ui

# 从本地存储加载上次设置的暗黑模式
ui.run_javascript('''
    const savedDarkMode = localStorage.getItem('darkMode');
    if (savedDarkMode !== null) {
        // 将本地存储的值同步到 NiceGUI 的 dark_mode 变量
        window.nicegui.dark_mode = savedDarkMode === 'true';
    }
''')

# 监听暗黑模式变化，保存到本地存储
def save_dark_mode_preference():
    ui.run_javascript(f'''
        localStorage.setItem('darkMode', {ui.dark_mode.value});
    ''')

ui.dark_mode.watch(save_dark_mode_preference)

# 切换开关
```

```
ui.switch("暗黑模式", value=ui.dark_mode)
ui.label("偏好设置会自动保存, 下次启动生效")

ui.run()
```

五、官方文档重点强调的注意事项

1. 启用时机与组件渲染顺序

- 建议在创建组件前启用暗黑模式, 避免组件先渲染为日间模式再切换 (可能出现闪烁);
- 若需动态切换, NiceGUI 会自动重新渲染组件, 无需手动刷新。

2. 与自定义 CSS 的优先级

- 暗黑模式的默认样式优先级: Quasar 内置暗黑样式 < `ui.colors` 配置 < 自定义 CSS (`:root.dark` 选择器);
- 若自定义样式未生效, 可添加 `!important` 强制覆盖 (如 `background-color: #334155 !important`)。

3. 第三方组件适配

- 非 NiceGUI/Quasar 原生的第三方组件可能不支持自动适配, 需通过 `:root.dark` 手动编写适配样式;
- 若第三方组件自身支持暗黑模式, 可通过监听 `ui.dark_mode` 变化触发其主题切换方法。

4. 性能考虑

- 暗黑模式切换时, NiceGUI 会批量更新组件样式, 性能优于手动遍历组件;
- 避免在 `ui.dark_mode.watch` 中执行复杂逻辑 (如大量 DOM 操作), 可能导致切换卡顿。

5. 系统主题检测的限制

- `follow_system()` 依赖浏览器的 `window.matchMedia('(prefers-color-scheme: dark)')` API, 部分旧浏览器可能不支持;
- 若应用运行在无浏览器环境 (如桌面端打包), `follow_system()` 可能失效, 建议提供手动切换开关。

六、图书管理系统实战场景示例

场景 1: 全局暗黑模式切换 (带持久化)

```
from nicegui import ui

# 加载本地存储的暗黑模式偏好
ui.run_javascript('''
    const saved = localStorage.getItem('library_dark_mode');
    if (saved) window.nicegui.dark_mode = saved === 'true';
''')

# 保存偏好到本地存储
def save_preference():
    ui.run_javascript(f'''
```

```

        localStorage.setItem('library_dark_mode', {ui.dark_mode.value});
    ''')
ui.dark_mode.watch(save_preference)

# 主题切换控件（集成到导航栏）
with ui.header(classes="justify-between items-center p-4"):
    ui.label("图书管理系统").classes("text-xl font-bold")
    ui.switch("暗黑模式", value=ui.dark_mode)

# 图书列表（自动适配暗黑模式）
with ui.card(classes="w-96 mx-auto p-6"):
    ui.label("热门图书").classes("text-lg font-bold mb-4")
    books = [
        {"title": "Python编程：从入门到实践", "author": "埃里克·马瑟斯"},
        {"title": "数据结构与算法", "author": "陈越"},
        {"title": "机器学习实战", "author": "彼得·哈灵顿"}
    ]
    for book in books:
        with ui.row(classes="justify-between items-center mb-2"):
            ui.text(f"《{book['title']}》")
            ui.text(book["author"], classes="text-secondary")

ui.run()

```

场景 2：自定义组件的暗黑模式适配

```

from nicegui import ui

# 自定义图书状态标签样式（支持暗黑模式）
ui.add_css('''
    .status-available {
        background-color: #dcfce7;
        color: #166534;
        padding: 2px 8px;
        border-radius: 4px;
        font-size: 0.8rem;
    }
    :root.dark .status-available {
        background-color: #166534;
        color: #dcfce7;
    }
    .status-borrowed {
        background-color: #fee2e2;
        color: #991b1b;
        padding: 2px 8px;
        border-radius: 4px;
        font-size: 0.8rem;
    }
    :root.dark .status-borrowed {
        background-color: #991b1b;
        color: #fee2e2;
    }
''')

```

```
'''  
  
# 切换开关  
ui.switch("暗黑模式", value=ui.dark_mode).classes("mb-4")  
  
# 图书状态展示  
with ui.card(classes="w-80 p-6"):  
    ui.label("图书状态").classes("text-lg font-bold mb-4")  
    with ui.row(classes="gap-4 mb-2"):  
        ui.text("《Python编程》")  
        ui.label("可借阅").classes("status-available")  
    with ui.row(classes="gap-4"):  
        ui.text("《数据结构》")  
        ui.label("已借出").classes("status-borrowed")  
  
ui.run()
```

七、总结

`ui.dark_mode` 是 NiceGUI 中实现暗黑模式的高效工具，其核心优势在于「零配置、响应式、高兼容」：

1. 无需手动编写大量适配样式，内置组件自动切换明暗主题；
2. 支持静态配置、动态切换、系统主题跟随三种模式，满足不同场景需求；
3. 可通过响应式监听和 CSS 选择器实现深度自定义适配；
4. 结合 `ui.colors`、CSS Layers 等特性，可构建统一且灵活的主题系统。

在图书管理系统开发中，合理使用 `ui.dark_mode` 可提升用户体验（如夜间办公时保护视力），同时降低主题维护成本，让开发重心集中在业务逻辑上。